

⚙ Roche DS Platform · AI-Powered Reliability

# Sentinel AI

Watch every pipeline service, then go from a raw failure to the exact root cause, the fix, and a one-click log deep-link — in **60 seconds**, not hours. No AWS expertise required.

● Built & running today

● Service Health monitoring

● Claude AI root-cause analysis

● GitHub Code Intelligence

# Every failure means the same slow, expert-only detective work

When `dp_monitor_db11224` lights up with FAILED records, an on-call engineer repeats four manual steps — each needing deep platform knowledge.

1

## Query Athena by hand

Write SQL, guess the time window, filter for FAILED — 10–20 min per investigation.

2

## Author CloudWatch queries from scratch

Must know log-group names, field names, error codes. One mistake = zero results.

3

## Read hundreds of log lines

Correlate trace IDs, device IDs, error codes by eye. High load, easy to miss the pattern.

4

## Write the RCA report

Document cause, impact, fix, escalation — another 30–60 min after the digging.

**1–3 hours of senior-engineer time per incident.** With concurrent failures, this is the single biggest drag on the on-call rotation.

# One tool, three audiences — zero prerequisite knowledge



## PLATFORM ENGINEER

### On-Call SRE

Paged at 2 AM for 50+ failing records. Needs the root cause *now* — without touching Athena or CloudWatch by hand.

Primary

High urgency



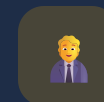
## QA / TEST ENGINEER

### QA Analyst

Validates pipeline health after each deploy. Confirms no new failure patterns crept in — with no deep AWS skills needed.

Post-deploy

Regression



## TECH LEAD / MANAGER

### Engineering Lead

Reads the executive summary at stand-up and shares a stakeholder-ready status — without opening an AWS console.

Reporting

Exec summary

**The barrier removed:** no need to know Athena SQL, CloudWatch Insights syntax, log-group paths, or which error codes map to which components. A junior engineer can now run a full RCA. The only prerequisite is read-only AWS credentials — supplied live in the app.

# A two-step web app that runs the whole investigation for you



## 1 · Fetch failures

Pick a range (1h / 1d / 1w or custom) and optional filters. Athena returns a paginated FAILED table in seconds.



## 2 · Analyze

Select the rows that matter, click **Analyze**. The AI hunts the logs and returns grouped root causes — no manual log digging.



## Failure groups

Each group: error pattern, root cause, category, impact count, immediate action, likely fix, affected files, escalation path.



## Executive summary

A 2–3 sentence, stakeholder-ready report generated automatically. Copy, paste into a ticket, done.



## One-click log deep-links

Every group carries a ready-made CloudWatch / Grafana link — right query, right log group, right time window.



## Recurring-issue memory

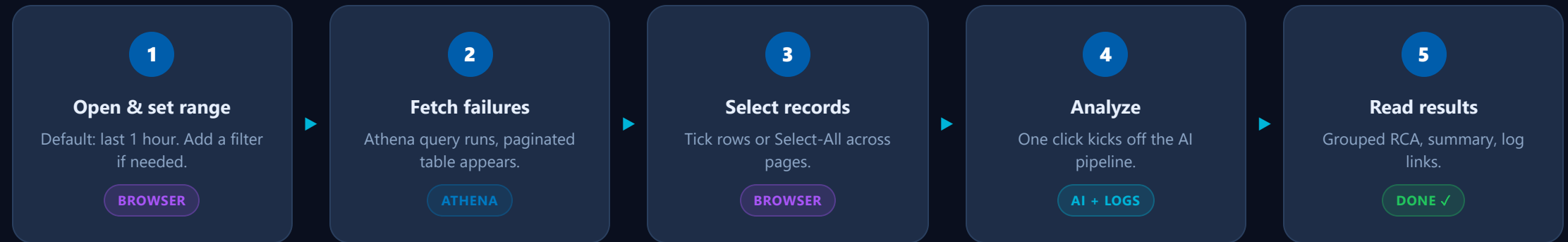
Repeat failures are auto-linked to a persistent RCA case, so "we've seen this before" is flagged for you.

**How to use it — literally 4 clicks:** open the app → choose a time range → **Fetch Failures** → tick rows → **Analyze Root Cause**. Read the grouped result, copy the summary, open the log link.

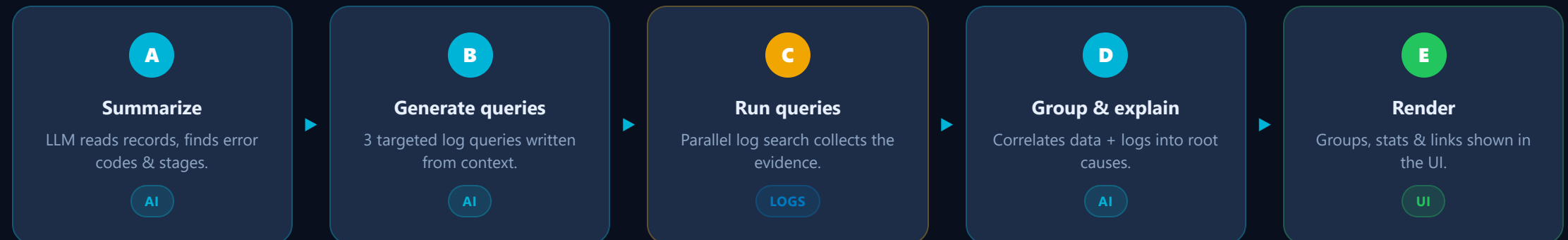
## HOW IT WORKS

# From alert to actionable RCA in under 60 seconds

### WHAT THE USER DOES



### WHAT THE SYSTEM DOES AUTOMATICALLY (BEHIND STEP 4)

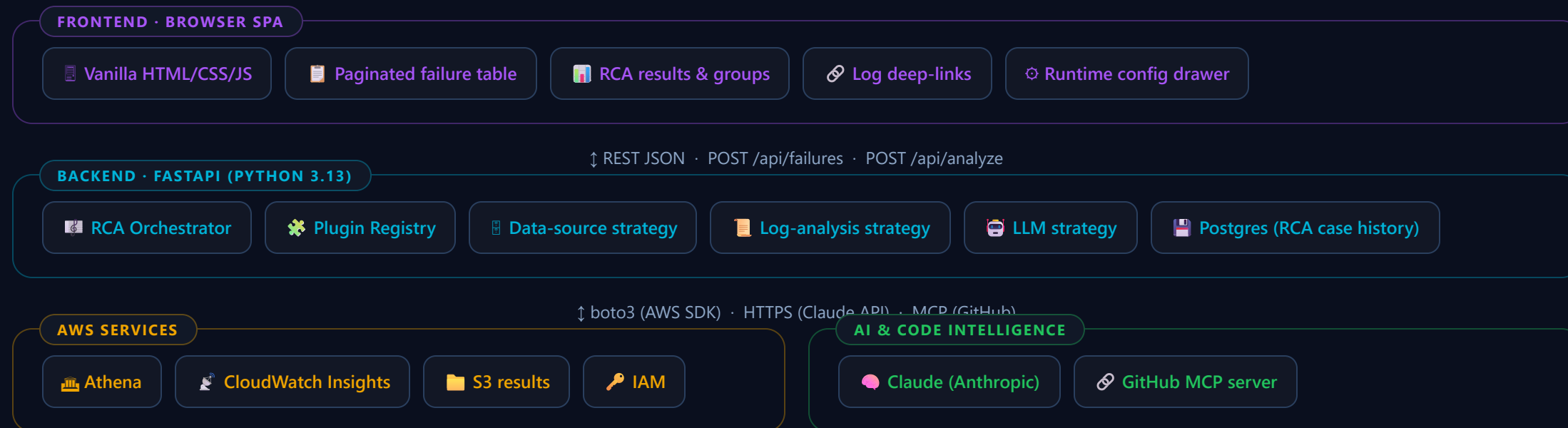


## ARCHITECTURE

# A clean three-layer design

Browser SPA → FastAPI orchestrator → pluggable AWS & AI backends.

Each layer swappable without touching the others.



# Strategy + Plugin Registry = swap any backend in one file



## Strategy pattern

The orchestrator depends only on three interfaces — `DataSource`, `LLM`, `LogAnalysis`. It never knows which concrete backend is running.



## Plugin registry

Providers are wired at request time from config. Athena↔local file, CloudWatch↔Grafana Loki, Claude↔any model — flipped by an env var.



## Open for extension

A new backend = one new provider file. The orchestrator and API routes never change. Zero regression risk.



## Testable by design

Local-file + fake-LLM providers mean the whole pipeline runs with no AWS, no keys — ideal for CI and demos.

```
# Orchestrator depends on abstractions only
class RCAOrchestrator:
    def __init__(self, data, llm, logs):
        self.data = data # DataSourceStrategy
        self.llm = llm # LLMStrategy
        self.logs = logs # LogAnalysisStrategy

# Registry picks concrete providers from .env
data = registry.data_source("athena")
llm = registry.llm("anthropic")
logs = registry.log_backend("cloudwatch")

# Swap a backend? Add ONE file. Nothing else changes.
```

# The LLM runs 5 precise jobs — each grounded in real failure data

①

## Summarize failures

Reads up to 50 records; identifies failing components, exact error codes, stage clustering, and time-based bursts.

②

## Generate log queries

Writes 3 targeted queries — error-code search, component + ERROR filter, and trace-ID correlation.

③

## Execute & collect

Runs the queries in parallel against the configured log backend and gathers the relevant log lines.

④

## RCA + grouping

Correlates records + logs into structured groups with real field values — no vague "check the logs".

⑤

## Executive summary

Distills all groups into a leadership-ready paragraph, ready to paste into an incident update.

⑥

## Guardrails

Prompts demand actual values (component, error\_code, stage, device\_id). Robust JSON parsing tolerates markdown fences.



BEYOND RCA · SOURCE CODE INTELLIGENCE

# From "why did it fail" to "is the fix any good"

A GitHub MCP integration brings repo, branch, and PR awareness into the same app — and an AI reviewer inspects the diff.



## Repo & PR browser

Lists repo info, branches, and pull requests via the GitHub MCP server — no console-hopping.



## AI code review

One click fetches a PR/branch diff and sends it to Claude for a structured, severity-ranked review.



## Related code changes

Surfaces the commits & PRs in the failure's time window — the likely change that broke things.

 Security & secrets

 SQL injection

 SonarQube-style smells

 Formatting

 Feature recommendations

# See every pipeline service at a glance — then drill into exactly why it failed



## Health at a glance

Glue jobs, Glue workflows, Lambda functions & DataSync tasks — each shows up / down and a failure count, colour-coded green → amber → red, with the total across all services.



## Monitoring profiles

Group resources into named profiles (e.g. "Prod ETL", "Ingestion") and switch between them — each team watches just their own pipelines.



## Any time window

Count failures over 1h / 24h / 7d or a custom date range — the same range carries straight into the drill-down.



## Run-history drill-down

Click a service to see its individual runs / invocations — success or failure — with timestamps and native error detail.



## Fetch failure reasons

One click pulls the CloudWatch log reasons behind the failures — the same detector that powers RCA, scoped to that resource.



## Zero console-hopping

All from read-only AWS credentials supplied in-app — no Glue, Lambda, or DataSync console access required.

**Reactive RCA, meet proactive monitoring:** RCA explains a failure after the alert — Service Health surfaces failing jobs, workflows, and functions on demand, and drills all the way down to the root-cause logs in the same app.

END-TO-END · HOW EVERY FEATURE FLOWS

# One app, three workflows — from setup to resolved incident

① **Setup (once):** save read-only AWS credentials, CloudWatch log groups, health-monitoring profiles, and a GitHub repo — all in the in-app Config tab.

## Service Health

Pick a profile + window



Up / down + failure counts



Drill into a service



Fetch CloudWatch reasons

## Root Cause Analysis

Fetch failures (Athena / CloudWatch)



Select records



AI analyzes the logs



Grouped causes + log deep-links

## Code Intelligence

Browse repo / PRs



Diff → AI code review



Severity-ranked findings



Related change in the failure window

**The payoff:** failures are caught early on the Health dashboard, root-caused in seconds by the AI pipeline, and the suspect code change is reviewed — without ever leaving the app or opening an AWS console.

WHY SENTINEL AI

# The advantages that matter to you

**~98%**

## Faster to root cause

1–3 hours of manual work collapses to under 2 minutes.

**0**

## Prerequisite knowledge

No Athena SQL, no CloudWatch syntax, no tribal knowledge.

**1**

## File to add a backend

Vendor-flexible by design — swap AWS, log source, or model.

**Live**

## Runtime config

Credentials, region, DB, and model changed in-app — no restart.



## Secure & auditable

Secrets are write-only in the UI (never returned to the browser). Every fetch is snapshotted; every RCA is persisted with a case history for a full audit trail.



## Consistent & repeatable

The same rigorous 5-step investigation runs every time — no steps skipped under pressure, no variation between engineers, day or night.

## IMPACT

# Time saved per incident

Measured against the manual 4-step RCA for a typical 50-record pipeline failure.

Manual: 15–30 min

**<30s**

Failure discovery

✓ Instant on click

Manual: 20–45 min

**<60s**

Log query + review

✓ Auto-generated

Manual: 30–60 min

**<90s**

Full RCA report

✓ Instant after Analyze

Manual investigation

1 – 3 hours

Sentinel AI (end-to-end)

60 – 120 seconds

~98% reduction in mean time to root cause — and the knowledge barrier is gone entirely.

## FUTURE POSSIBILITIES

# Built today · designed to grow

The strategy/plugin architecture means each item below is an add-on, not a rewrite.

DONE

### AI RCA · service health monitoring · runtime config · direct Claude · code intelligence

The investigation pipeline, the proactive Glue/Lambda/DataSync health dashboard with profiles, live configuration, and GitHub code review are shipped and running.

---

NEXT

### Auto-link RCA → PR & inline review comments

Surface the pull request that fixes a failure, and post AI findings straight onto the PR at the right file/line.

---

NEXT

### Full repository scan & failure-trend dashboard

Run the review checklist across the whole repo; aggregate stored snapshots into trends, top error codes, and MTTR metrics.

---

VISION

### Integration hub — Jira · Slack · Email

Auto-raise tickets, notify channels, and alert stakeholders when failures cross a threshold. No copy-paste.

---

VISION

### Predictive failure detection & auto-remediation

Learn from history to flag components likely to fail — and one-click re-submit failed records back into the pipeline.



ROCHE DS PLATFORM · SENTINEL AI

# From failure to fix in 60 seconds.

Built with FastAPI · Amazon Athena · CloudWatch Logs Insights ·  
Glue / Lambda / DataSync health · Claude AI · GitHub MCP

● ~98% faster RCA

● Proactive service health

● Pluggable & future-ready

● Running today